Problem Challenge 2: Path with Maximum Sum (hard)

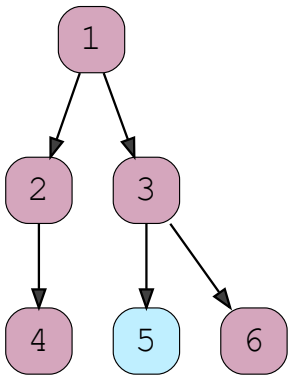
Problem Statement

Find the path with the maximum sum in a given binary tree. Write a function that returns the maximum sum.

A path can be defined as a **sequence of nodes between any two nodes** and doesn't necessarily pass through the root. The path must contain at least one node.

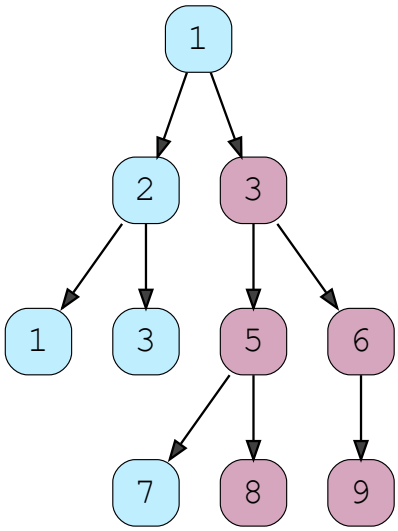
Example 1:

Output: 16
Explanation: The path with maximum sum is: [4, 2, 1, 3, 6]



Example 2:

Output: 31
Explanation: The path with maximum sum is: [8, 5, 3, 6, 9]



Try it yourself

Try solving this question here:

Java

```java
/*class TreeNode {
  int val;
  TreeNode left;
  TreeNode right;

  TreeNode(int x) {
    val = x;
  }
};*/

class Solution {

  private static int globalMaximumSum;

  public int findMaximumPathSum(TreeNode root) {
    globalMaximumSum = Integer.MIN_VALUE;
    // TODO: Write your code here
    return globalMaximumSum;
  }
}

```

Test Case | Run Code Result | Stdout

```
[1, 2, 3]
```

Use Example Testcases | Run | Submit

← Previous

Solution: Problem Challenge 1: Tree Diameter

Next →

Solution: Problem Challenge 2: Path With Maximum Sum

Mark as Completed